OPEN PLATFORM
FIRMWARE (OPF)

openSFI Update: Advancing Silicon Firmware Interoperability

Raj Kapoor – Fellow, Chief Firmware Architect – **AMD Inc.**

Albert Rennie – Principal Engineer – **AMD Inc.**

Allison Goodman

OCP PLATINUM Member

Why openSFI? | Rethinking the Host FW-Silicon Contract

- Platform's Host Firmware today is tightly coupled to silicon implementations
- Host Firmwares embed silicon-specific knowledge
- Limited platform firmware portability across silicon vendors
- Duplication of effort by platform integrators across ecosystem

Vision

- Define a vendor-agnostic interface between host firmware and silicon firmware
- Enable portable host firmware across vendors
- Encapsulate silicon complexity behind a clean abstraction
- Foster ecosystem-scale collaboration

From Vision to Execution: Iterative Progress

Q1 2025 (F2F) → kick off openSFI v0.3 Specification Iteration

- Established initial interface model
- Defined early architectural boundaries

Q3 2025 (Global OCP Summit) → Ratify & Release openSFI v0.3 Specification

Q3 2025 (F2F) → kick off openSFI v0.5 Specification Iteration

- Introduced clearer, consistent stage-based initialization phases
- Improved alignment across x86 **(& now ARM!!)** silicon vendors

Outcome

- Converging toward a testable and implementable model
- Foundation for PoC (Vendor Abstraction Layer implementation) across the 3 Si. Vendors

A Shared, Stage-Based Initialization Flow

Alignment across multiple silicon vendors

Common understanding of:

- Silicon initialization stages
- Configuration and Capabilities available per stage
- Interface boundaries

Key Principle:

- Host firmware orchestrates and owns system resources
- Silicon firmware executes

Benefits:

- Reduced iterative dependencies and ambiguity
- Improved portability of host software- minimal changes to get to 1st boot on new silicon
- Easier integration with cleaner separation of compute silicon from platform

v0.5 Spec Deltas | High-Level API & Stage Summary

1. Modified Init Silicon API

Consolidates arguments to simplify the initialization entry point.

v0.3 Parameters (Deprecated)

`Input` / `Output` Stage buffers

`Stage` Initialization stage identifier

`SoCID` Vendor-specific SoC ID

v0.5 Parameters (Updated)

`SFIInput` / `SFIOutput` Contiguous buffers

`InitStageID` Unified stage identifier

2. Modified Runtime Op API

Transitions from custom runtime operations to structured Post-Silicon Init ops.

v0.3 Runtime Op

Relied on `ExecuteSiliconRuntimeOp` with raw stage and vendor SoC identifiers.

v0.5 Post-Silicon Op

Introduces `ExecutePostSiliconInitOp` which operates on contiguous SFI buffers.

Targets specific `PostInitOpsID` for standardized post-initialization tasks.

3. Added Silicon Init Stages

Provides structured milestones throughout the boot sequence.

MEM_INIT: Exposes a stable memory map.

SI_RECONFIG: Applies host config decisions.

POST_PCI: Resource map dependent silicon config.

PREPARE_OS_HANDOFF: Pre-boot coordination.

EXIT_HOST_FW: Final firmware execution under host control.

v0.5 spec - openSFI Phases | OSIP Stage Summary

1. Memory Init (MIP)

OSIP.SRM Phase

Return initial system resource maps including memory

Key Functions:

- Invokes `InitializeSilicon()` with `SFI_STAGE_GET_SYS_RSC_MAP`
- Establishes initial stack and heap availability
- Ready for executing from C main

2. Address Resolution (ARP)

OSIP.ARP Phase

Discovers and allocates platform resources like PCI enumeration & address assignment to converge platform & SoC configs

Key Functions:

- Invokes `InitializeSilicon()` with `SFI_STAGE_SI_RECONFIG`
- Resolves resource conflicts iteratively
- Can request reset via `SFI_STAGE_RESET_REQUEST`

3. Post-PCI Init (PPI)

OSIP.PPI Phase

Executes silicon configuration steps dependent on finalized resource mappings.

Key Functions:

- Invokes `InitializeSilicon()` with `SFI_STAGE_POST_PCI_INIT`
- Programs silicon using allocated addresses
- Configures hardware units & enables peripheral devices

4. OS Launch (OSL)

OSIP.OSL Phase

The final stage of OSIP ensuring readiness for OS execution.

Key Functions:

- Invokes `InitializeSilicon()` with `SFI_STAGE_PREPARE_FOR_OS_HANDOFF`
- Invokes final call with `SFI_STAGE_EXIT_HOST_FW`
- Signals completion of host firmware responsibilities
- Transfers control to OS bootloader

v0.5 spec - openSFI Data Record Format

Plan

Align with Firmware Handoff Spec's transfer-list/record model as a reference for passing configuration data from silicon init → PEP → host firmware.

Adoption pending contribution to OCP.

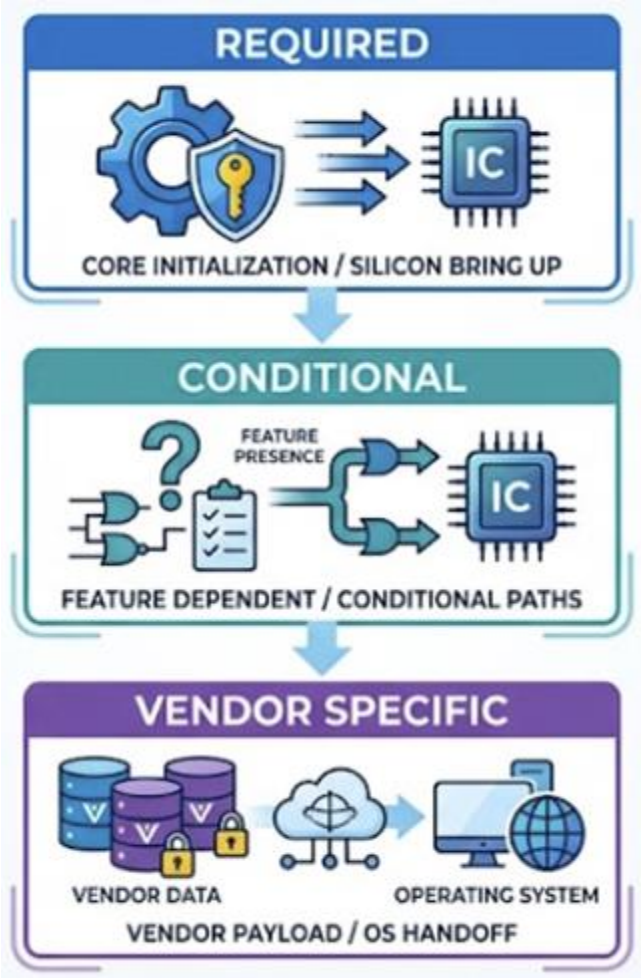
Key Decisions

- Use Firmware Handoff Spec's 32-bit tag ID / sub-record structure as reference model
- Single unified transfer list (not separate input/output) to reduce copy overhead
- openSFI-specific naming only for structures unique to openSFI; reuse standard names where structures overlap with existing specs

Open Items

- Reserve a tag-ID range for openSFI (24 bits available, unallocated)
- Non-volatile storage ownership for persisting transfer-list data across reboots
- Single unified transfer list vs. separate input/output lists
- Binary encoding of records
- Common cross-vendor input/output definitions

v0.5 spec - openSFI Data Record Classifications



Required

- Required for silicon bring up
- Core requirement for initialization

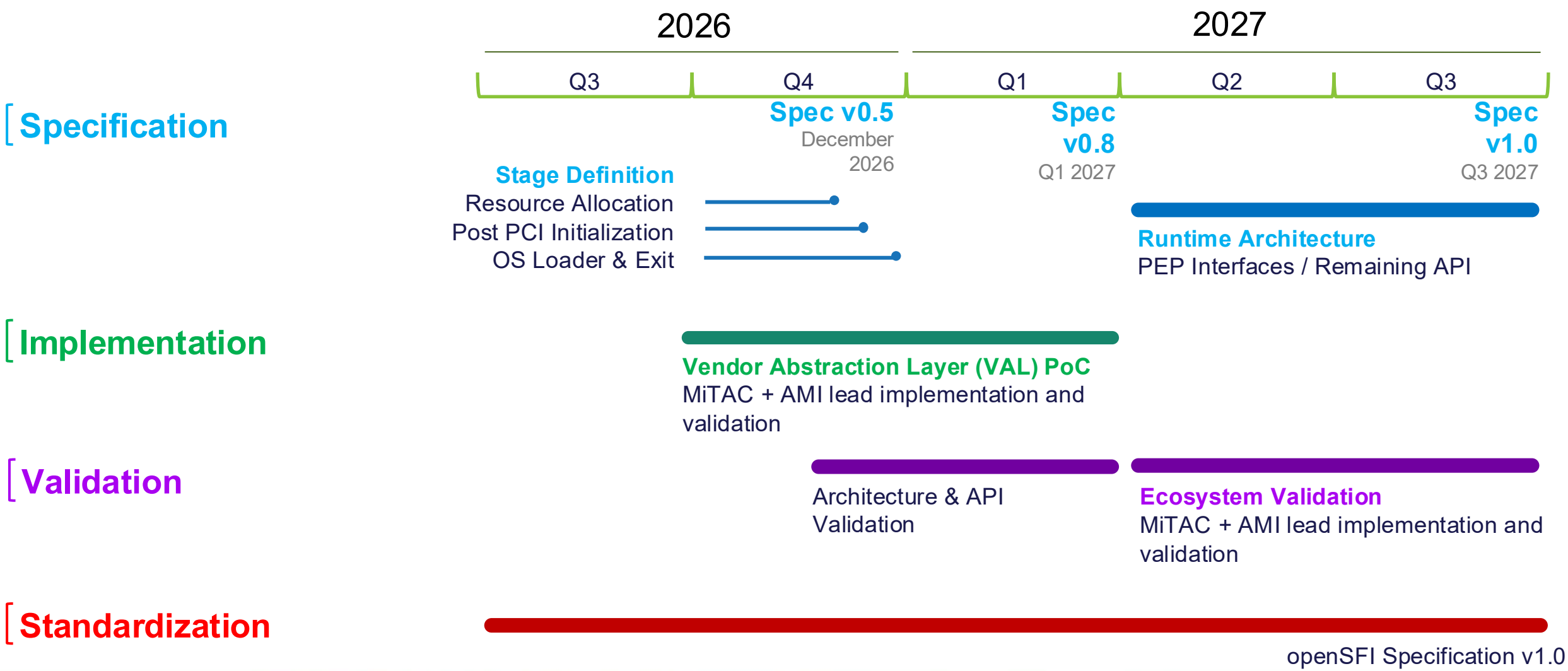
Conditional

- Required conditionally upon feature presence
- No feature support, no data required

Vendor Specific

- Must not be required for full silicon operation
- Not defined by openSFI
- Blob/Payload passed to operating system

openSFI Development Roadmap (Proposed Targets)



The openSFI Ecosystem Continues To Grow!!





9ELEMENTS















Hewlett Packard
Enterprise











OPEN
SOURCE
FIRMWARE
FOUNDATION



Compliance with OCP Tenets

Openness

Promotes openness and efficiency by enabling third parties to build, modify, and personalize a unified firmware interface that is compatible across multiple platforms and silicon architectures.

Efficiency

Reduce operational expenses, capital expenses and carbon emissions through a unified firmware interface. This enables efficient resource utilization, streamlines development processes, and eliminates redundant engineering efforts.

Impact

Has transformative impacts on the industry by accelerating time-to-market with standardized interfaces.

Sustainability

Enables optimization of code to reduce resource consumption.

Call to Action

Participate in openSFI

- Contribute to v0.5 refinement
- Engage in PoC development

Collaborate Across the Ecosystem

- Silicon, platform integrator, hyperscaler, and firmware partners
- Open-source and standards communities

Align for the Future

- Interoperable firmware across vendors
- Alignment with emerging security and regulatory expectations (e.g., EU CRA)

Thank You!

